

Sistema Web Empresarial de Gestión de Ventas y Analítica Estadística

Fases 01 – 07 del Plan Integral de Desarrollo

Documento detallado del trabajo ejecutado: estado del plan de 16 fases, decisiones de arquitectura, referencia completa de la API, guía de arranque, pruebas realizadas, hallazgos corregidos y roadmap pendiente.

Documento	Documentación de Avance por Fases
Versión	1.0
Fecha	04 de octubre de 2026
Estado	Fases 01–07 completadas · Fase 08 en curso
Stack	React + TypeScript · Python/FastAPI · PostgreSQL
Commits	c8162f0 · 6cc70d6 · 1a3f263 · 0d4286d

Índice

Fases 01-07 del Plan Integral de Desarrollo	3
1. Resumen ejecutivo	3
2. Estado del plan maestro	3
3. Fase por fase: qué se hizo	4
3.1 Paso previo — Arreglos base (commit c8162f0)	4
3.2 Fase 05 (porción) + Fase 07 — Backend CRUD (commit 6cc70d6)	5
3.3 Fase 06 (porción) — Frontend React (commit 6cc70d6)	6
3.4 Fase 14 (inicio) — Pruebas	7
4. Decisiones de arquitectura	7
5. Guía de arranque	8
Requisitos	8
Backend	8
Frontend	8
Base de datos (cuando esté PostgreSQL)	8
Credenciales de desarrollo	8
Comandos de calidad	9
6. Referencia de API (actual)	9
Sistema	9
Autenticación	9
Clientes	10
Productos	10
Categorías	10
7. Estructura del repositorio	12
8. Hallazgos y correcciones durante el avance	13
9. Próximos pasos (roadmap inmediato)	13
10. Criterios de aceptación verificados en esta etapa	14

Fases 01-07 del Plan Integral de Desarrollo

Campo	Detalle
Documento	Documentación técnica de avance por fases
Versión	1.0
Fecha	04 de octubre de 2026
Estado	Fases 01-07 completadas · Fase 08 en curso
Estructura	Monorepo: <code>frontend/</code> · <code>backend/</code> · <code>database/</code> · <code>docs/</code>
Commits	<code>c8162f0</code> (paso 1) · <code>6cc70d6</code> (paso 2) · reorganización en <code>frontend/</code>

1. Resumen ejecutivo

Se trabajó sobre las fases tempranas del plan maestro de 16 fases. El punto de partida tenía una base de datos bien definida y autenticación JWT, pero **ningún módulo funcional**: la API solo tenía login, y el frontend era un shell con un solo dashboard de datos falsos y 11 páginas "próximamente".

Al cerrar este avance, el sistema cuenta con:

- **API REST funcional** con CRUD completo de clientes, productos y categorías.
- **Frontend con enrutamiento real**, login protegido, cliente API con manejo de sesión y páginas CRUD operativas.
- **Suite de pruebas automatizadas** (4 tests de render) y verificación end-to-end de 23 comprobaciones en backend.
- **Entorno de desarrollo configurado** (variables de entorno, proxy Vite→FastAPI, venv, seeds).

2. Estado del plan maestro

Fase	Nombre	Estado	Observación
01	Análisis y levantamiento	✓ Completada	El documento <i>Plan Integral de Desarrollo</i> cumple el entregable (requisitos RF-01...RF-22, actores, criterios de aceptación).
02	Arquitectura técnica	✓ Completada	Frontend/API/PostgreSQL separados; contratos REST definidos en código (ver §6); estructura de carpetas conforme al plan.

Fase	Nombre	Estado	Observación
03	UX/UI empresarial	✓ Completada	Sistema de diseño azul/cyan existente; sidebar, topbar, dashboard, tablas, estados <i>loading/empty/error</i> y sistema de modales/formularios extendido.
04	Base de datos PostgreSQL	✓ Completada*	<code>001_initial_schema.sql</code> (24 tablas + índices) y seeds. *Falta ejecutar contra un servidor PostgreSQL (ver §10).
05	Backend/API	● En curso (≈45%)	Auth JWT ✓ · CRUD clientes/productos/categorías ✓ · pendiente: ventas, inventario, estadística, RBAC, auditoría.
06	Frontend React	● En curso (≈50%)	Router, layout, API client, login, estados de carga/error ✓ · pendiente: módulos Ventas/Analytics/Insights/Reportes.
07	Clientes y productos	✓ Completada	CRUD backend + frontend, búsqueda, filtros, historial por empresa, relación producto↔categoría.
08	Ventas, pedidos e inventario	● Iniciada	Creado <code>backend/app/schemas/sale.py</code> . Servicio transaccional y UI pendientes.
09	Motor estadístico (Semana 07)	● Pendiente	
10	Dashboard Analytics	● Parcial	UI existente con datos ilustrativos; sin KPIs reales.
11	Insights empresariales	● Pendiente	
12	Reportes	● Pendiente	
13	Seguridad y auditoría	● Parcial	Auth JWT ✓ · pendiente RBAC, permisos por rol y registro de auditoría.
14	Pruebas y calidad	● Iniciada	4 tests de render + smoke test CRUD de 23 aserciones.
15	Despliegue	● Pendiente	
16	Cierre y documentación	● Iniciada	Este documento. README aún es el de plantilla.

3. Fase por fase: qué se hizo

3.1 Paso previo — Arreglos base (commit `c8162f0`)

Correcciones que habilitaron el desarrollo:

Cambio	Archivo	Motivo
Variables de entorno reales	<code>backend/.env</code> (nuevo, no versionado)	El backend no arrancaba sin <code>DATABASE_URL</code> y <code>JWT_SECRET_KEY</code> .
<code>.env.example</code> en formato plano	<code>backend/.env.example</code>	Venía como snippet de PowerShell (<code>@' ... '@ \[Set-Content</code>), no utilizable en Linux.
Proxy del dev server	<code>vite.config.ts</code>	<code>POST /api/*</code> , <code>/docs</code> y <code>/openapi.json</code> se redirigen a <code>http://localhost:8000</code> . El frontend usa rutas relativas (<code>/api/...</code>), sin CORS en desarrollo.
<code>__init__.py</code> añadidos	<code>backend/app/{api,services,schemas}/</code>	Paquetes incompletos (funcionaban por namespace packages).
<code>Base</code> duplicada eliminada	<code>backend/app/db/session.py</code>	Existía una segunda clase <code>Base</code> no usada; los modelos usan <code>app.models.base.Base</code> .
Servicio vacío eliminado	<code>backend/app/services/auth.py</code>	Duplicado de <code>auth_service.py</code> (0 líneas).
Código muerto eliminado	<code>src/App.css</code> , <code>src/assets/vite.svg</code> , <code>src/assets/react.svg</code> , <code>public/icons.svg</code>	Restos de la plantilla Vite, sin referencias.
Favicon corregido	<code>index.html</code>	Apuntaba a <code>href="/"</code> ; ahora <code>/favicon.svg</code> .
Seed de administrador	<code>database/seeds/002_admin_user.sql</code>	Sin usuario no era posible iniciar sesión.
Entorno Python	<code>backend/.venv/</code>	<code>python3 -m venv</code> + <code>pip install -r requirements.txt</code> .

3.2 Fase 05 (porción) + Fase 07 — Backend CRUD (commit `6cc70d6`)

Patrón de capas implementado (se repetirá en fases siguientes):

```

api/<recurso>.py      → router FastAPI: validación de entrada, códigos HTTP, auth
    ↓
services/<recurso>_service.py → lógica de negocio, consultas SQLAlchemy, paginación
    ↓
models/<recurso>.py   → modelos SQLAlchemy 2.0 (Mapped[])
    ↓
PostgreSQL (migración 001)

```

Módulos creados

Módulo	Router	Service	Schemas
Clientes	<code>backend/app/api/customers.py</code>	<code>customer_service.py</code>	<code>schemas/customer.py</code>

Módulo	Router	Service	Schemas
Productos	<code>backend/app/api/products.py</code>	<code>product_service.py</code>	<code>schemas/product.py</code>
Categorías	<code>backend/app/api/categories.py</code>	<code>category_service.py</code>	<code>schemas/category.py</code>
Común	—	—	<code>schemas/common.py</code> (<code>Page<T></code> paginado)

Reglas de negocio implementadas

- Aislamiento por empresa:** toda consulta filtra por `company_id` del usuario autenticado. Un usuario jamás ve datos de otra empresa (verificado con prueba de aislamiento).
- Borrado lógico:** `DELETE` desactiva (`is_active = false`), nunca borra filas (las FK lo impedirían y se preserva la trazabilidad).
- Únicos compuestos** (devueltos como HTTP 409): - Clientes: (`company_id, document_type, document_number`) — los `NULL` no colisionan (permitido repetir clientes sin documento). - Productos: (`company_id, sku`). - Categorías: (`company_id, name`) (comparación sin importar mayúsculas).
- Validación de categoría ajena** al crear/editar producto → HTTP 400.
- Paginación** con `page`, `page_size` (máx. 100) y `total`.
- Búsqueda** parcial por nombre, correo, documento, teléfono (clientes) o nombre, SKU, descripción (productos), usando `ILIKE`.
- Precios** validados con `ge=0` y `max_digits=12` (coincide con el `CHECK` de la migración).

Auditoría de seguridad de tipos: se detectó que el validador Pydantic rechaza `@salesia.local` (dominio reservado), lo que habría dejado el sistema sin login posible. Los seeds usan `admin@salesia.com`.

3.3 Fase 06 (porción) — Frontend React (commit `6cc70d6`)

Arquitectura creada (dentro de `frontend/`)

```
frontend/src/
├── App.tsx                → BrowserRouter + AuthProvider + rutas
├── navigation.ts          → mapa rutas/íconos/títulos del sidebar
├── lib/api.ts             → cliente HTTP centralizado
├── types/index.ts         → tipos compartidos (Customer, Product, Category, Page<T>)
├── context/
│   ├── auth-context.ts    → contexto + hook useAuth
│   ├── AuthContext.tsx    → AuthProvider (token, /auth/me, login/logout)
│   ├── search-context.ts  → contexto + hook useSearch
│   └── SearchContext.tsx  → SearchProvider (búsqueda global topbar)
├── layouts/AppLayout.tsx  → shell (sidebar + topbar + Outlet + footer)
├── components/            → Sidebar, Topbar, MetricCard, Modal
├── pages/                 → Login, Dashboard, Customers, Products
└── utils/format.ts        → moneda es-PE/PEN, iniciales, fechas
```

Funcionalidad

- **Rutas protegidas:** sin sesión → redirect a `/login`; con sesión → no se puede volver al login. Si el token expira (401 fuera del login), se limpia y redirige.
- **Cliente API** (`lib/api.ts`): token en `Authorization`, parseo de errores de Pydantic (lista de `detail`), mensajes en español, timeout de conexión amigable (`"No se pudo conectar con el servidor"`).
- **Páginas CRUD:** tabla con estados *cargando* / *error* / *vacío*, búsqueda con *debounce* de 250 ms, paginación con auto-corrección de página fuera de rango, modales de creación/edición, confirmación de borrado, checkbox "incluir inactivos".
- **Productos:** pestañas *Productos* / *Categorías*, filtro por categoría, precios formateados en soles.
- **Navegación:** los módulos no implementados (Ventas, Analytics, Insights, Inventario, Pagos, Reportes, Configuración, Ayuda) muestran un placeholder descriptivo con el alcance de su fase.
- **Diseño:** se añadieron estilos para login, formularios, modales, pestañas, estados de tabla, paginación y spinner, siguiendo las variables CSS existentes.

3.4 Fase 14 (inicio) — Pruebas

Prueba	Herramienta	Cubre
<code>src/App.test.tsx</code> (4 tests)	Vitest + Testing Library + jsdom	Redirect a login, login correcto → dashboard, login incorrecto → mensaje de error, rutas privadas protegidas.
Smoke test CRUD (23 aserciones)	pytest-compatible (script con TestClient)	Login, <code>/me</code> , 401 sin token, CRUD completo, duplicados → 409, precios negativos → 422, paginación, borrado lógico, aislamiento entre empresas.

Comandos: `cd frontend && npm test` · `backend/.venv/bin/python <smoke>` (backend).

4. Decisiones de arquitectura

1. **Prefijo `/api` en todos los endpoints** — consistente con `/api/auth` existente y con el proxy de Vite.
2. **JWT en `localStorage`** — simple y suficiente para el alcance académico; se evaluará *refresh token* en la Fase 13.
3. **Sin `relationship()` en los modelos** — se usan `JOIN` explícitos (`select(Product, Category.name)`) para mantener el control de las consultas. Se agregará `relationship()` cuando convenga.
4. **Precio siempre del catálogo** — el cliente no envía precios; el servidor lee `products.unit_price` (evita manipulación).
5. **Datetime con zona horaria en Python** (`datetime.now(timezone.utc)`) — la migración usa `TIMESTAMP TZ`.
6. **Errores de negocio como HTTP status semánticos:** 400 (dato inválido), 401 (auth), 404 (no encontrado), 409 (conflicto de únicos), 422 (validación Pydantic).

7. **Sin Alembic todavía** — la fuente de verdad es `database/migrations/001_initial_schema.sql`; se evaluará Alembic al iniciar la Fase 15.

5. Guía de arranque

Requisitos

- Python 3.12+ · Node **22** (`nvm use 22` — Vite 8 no funciona con Node 18) · PostgreSQL 16 (servidor, aún no instalado)

Backend

```
cd backend
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/uvicorn app.main:app --reload --port 8000
```

Documentación interactiva: <http://localhost:8000/docs>

Frontend

```
cd frontend
nvm use 22
npm install
npm run dev          # http://localhost:5173 (proxy a :8000)
```

Base de datos (cuando esté PostgreSQL)

```
sudo apt install postgresql-16          # sólo cliente instalado hoy
psql -U postgres -c "CREATE ROLE salesia_app WITH LOGIN PASSWORD '7E_VrIOYGXtyr4nN0BVo0CA8';"
psql -U postgres -c "CREATE DATABASE salesia_enterprise OWNER salesia_app;"
psql -U salesia_app -d salesia_enterprise -f database/migrations/001_initial_schema.sql
psql -U salesia_app -d salesia_enterprise -f database/seeds/001_reference_data.sql
psql -U salesia_app -d salesia_enterprise -f database/seeds/002_admin_user.sql
```

Mientras no haya PostgreSQL se usó un lanzador temporal con SQLite (`/tmp/opencode/run_backend_sqlite.py`) que replica el esquema y siembra datos de prueba. No forma parte del repositorio.

Credenciales de desarrollo

Campo	Valor
Correo	<code>admin@salesia.com</code>

Campo	Valor
Contraseña	Admin123*
Rol	Administrador

⚠ Cambiar en cualquier entorno que no sea local.

Comandos de calidad

```
cd frontend
npm run lint      # ESLint
npm run build     # tsc -b + vite build
npm test          # Vitest (4 tests)
```

Nota: si `npm install` vuelve a sufrir el bug de dependencias opcionales de npm (`Cannot find native binding` de rolldown), ejecutar desde `frontend/`: `npm i --no-save @rolldown/binding-linux-x64-gnu@$(node -p "require('./node_modules/rolldown/package.json').version")`

6. Referencia de API (actual)

Base URL desarrollo: `http://localhost:5173` (proxy) o `http://localhost:8000` (directo).

Sistema

Método	Ruta	Auth	Descripción
GET	/	—	Info de la API
GET	/health	—	Salud general
GET	/health/database	—	Conexión a PostgreSQL

Autenticación

Método	Ruta	Auth	Descripción
POST	/api/auth/login	—	{email, password} → {access_token, token_type}
GET	/api/auth/me	Bearer	Perfil del usuario autenticado

Cientes

Método	Ruta	Descripción
GET	<code>/api/customers? search=&page=1&page_size=20&include_inactive=false</code>	Lista paginada <code>{items, total, page, page_size}</code>
GET	<code>/api/customers/{id}</code>	Detalle (404 si no existe)
POST	<code>/api/customers</code>	Crea (201; 409 si documento duplicado)
PATCH	<code>/api/customers/{id}</code>	Edición parcial
DELETE	<code>/api/customers/{id}</code>	Desactiva (borrado lógico)

Productos

Método	Ruta	Descripción
GET	<code>/api/products? search=&category_id=&page=&page_size=&include_inactive=</code>	Lista paginada con <code>category_name</code>
GET	<code>/api/products/{id}</code>	Detalle
POST	<code>/api/products</code>	Crea (201; 409 SKU duplicado; 400 categoría inexistente)
PATCH	<code>/api/products/{id}</code>	Edición parcial
DELETE	<code>/api/products/{id}</code>	Desactiva

Categorías

Método	Ruta	Descripción
GET	<code>/api/categories?search=&include_inactive=</code>	Lista completa
GET	<code>/api/categories/{id}</code>	Detalle
POST	<code>/api/categories</code>	Crea (201; 409 nombre duplicado)
PATCH	<code>/api/categories/{id}</code>	Edición parcial
DELETE	<code>/api/categories/{id}</code>	Desactiva

Ejemplo

```
TOKEN=$(curl -s -X POST localhost:8000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"admin@salesia.com","password":"Admin123*"}' | jq -r .access_token)

curl -s -H "Authorization: Bearer $TOKEN" \
"localhost:8000/api/customers?search=maría" | jq
```

7. Estructura del repositorio

```

salesia-enterprise/
├── .gitignore                                # raíz (node_modules, dist, .venv, .env...)
├── README.md                                # arranque rápido y estado del proyecto
├── frontend/                                # FRONTEND (React + TS + Vite)
│   ├── .gitignore
│   ├── index.html                          # HTML base (favicon corregido)
│   ├── package.json                        # scripts: dev, build, lint, test, preview
│   ├── package-lock.json
│   ├── vite.config.ts                      # proxy /api → :8000
│   ├── vitest.config.ts                   # entorno jsdom para tests
│   ├── tsconfig.json · tsconfig.app.json · tsconfig.node.json
│   ├── eslint.config.js
│   ├── public/favicon.svg
│   ├── dist/                              # build de producción (no versionado)
│   ├── node_modules/                      # no versionado
│   └── src/
│       ├── App.tsx                        # router + guards
│       ├── App.test.tsx                  # tests de render
│       ├── navigation.ts
│       ├── index.css                      # sistema de diseño completo
│       ├── lib/api.ts                     # cliente HTTP
│       ├── types/index.ts
│       ├── context/                       # auth y búsqueda global
│       ├── components/                    # Sidebar, Topbar, MetricCard, Modal
│       ├── layouts/AppLayout.tsx
│       ├── pages/                         # Login, Dashboard, Customers, Products
│       ├── utils/format.ts
│       └── test/setup.ts
├── backend/                                # BACKEND (FastAPI)
│   ├── .env                               # no versionado (gitignored)
│   ├── .env.example
│   ├── .venv/                             # no versionado
│   ├── requirements.txt
│   └── app/
│       ├── main.py                        # app, CORS, registro de routers
│       ├── api/                           # auth, customers, products, categories, dependencies
│       ├── core/                          # config, security (JWT + argon2)
│       ├── db/                            # session (engine, get_db)
│       ├── models/                        # 25 modelos SQLAlchemy
│       ├── schemas/                       # common, auth, customer, product, category, sale*
│       └── services/                      # auth, customer, product, category
├── database/
│   ├── migrations/001_initial_schema.sql   # 24 tablas
│   ├── seeds/001_reference_data.sql        # roles, empresa, categorías, métodos de pago
│   └── seeds/002_admin_user.sql            # usuario administrador
└── docs/
    └── Documentacion_Avance_Fases_01-07.md # este documento

```

* sale.py creado, aún sin router/service.

8. Hallazgos y correcciones durante el avance

#	Hallazgo	Impacto	Resolución
1	<code>admin@salesia.local</code> rechazado por el validador de email (dominio reservado <code>.local</code>)	Login imposible con el seed	Emails cambiados a <code>admin@salesia.com</code> en ambos seeds
2	Un 401 en <code>/auth/login</code> (contraseña incorrecta) se trataba como "sesión expirada" y redirigía	Usuario no veía el error real	<code>lib/api.ts</code> distingue el intento de login del resto de rutas
3	Node 18 no soporta Vite 8 (<code>styleText</code> requiere Node ≥ 20)	<code>npm run build</code> fallaba	Usar <code>nvm use 22</code> ; documentado
4	Bug de npm: falta <code>@rolldown/binding-linux-x64-gnu</code>	<code>npm run build</code> fallaba	Instalar el binding exacto de la versión de rolldown
5	Modelos usan tipos PostgreSQL (<code>INET</code> , <code>JSONB</code>)	Imposible probar con SQLite	Compiladores de tipo registrados sólo en el script temporal de pruebas
6	PK <code>BigInteger</code> no autoincrementa en SQLite	Pruebas locales	Sólo afecta al entorno temporal de pruebas
7	Lint de React: <code>setState</code> síncrono dentro de <code>useEffect</code> y fast-refresh por hooks exportados junto a componentes	CI/rojo	Contextos separados (<code>*-context.ts</code> + <code>*.tsx</code>), resets de página en manejadores y auto-corrección asíncrona

9. Próximos pasos (roadmap inmediato)

- Fase 08 — Ventas, pagos e inventario** (*en curso*) - Servicio transaccional: venta → detalle → pago → descuento de inventario → movimiento de salida, en una sola transacción. - Endpoints `GET /api/sales`, `GET /api/sales/{id}`, `GET /api/inventory`, `POST /api/inventory/movements`, `GET /api/payment-methods`. - Páginas `VentasPage`, `InventarioPage`, `PagosPage`.
- Fase 09 — Motor estadístico**: media, mediana, comparación, variables, probabilidades y Teorema de Bayes (`statistics/`, `probability/`), persistiendo en `statistical_analyses/` / `statistical_results`.
- Fase 10-11 — Analytics + Insights**: `GET /api/dashboard/summary`, KPIs reales, insights deterministas con evidencia numérica.
- Fase 13 — Seguridad y auditoría**: `require_role`, permisos por rol, escritura en `audit_logs`.
- Fase 12/14/15/16**: reportes, ampliación de pruebas, despliegue y manuales.
- Infraestructura**: instalar servidor PostgreSQL y ejecutar migración + seeds (§5).

10. Criterios de aceptación verificados en esta etapa

Criterio (plan §20)	Estado
Un usuario autorizado puede iniciar sesión y acceder únicamente a sus módulos	✓
Las operaciones críticas quedan aisladas por empresa	✓
Validación de datos en frontend y backend (RNF-02)	✓
Autenticación segura con token (RNF-04, parcial)	✓
API documentada en <code>/docs</code> (RNF-05)	✓
Diseño responsive escritorio/tablet (RNF-06)	✓
Configuración por variables de entorno (RNF-10)	✓
Manejo centralizado de errores de API (RNF-11)	✓
Pruebas unitarias/de aceptación mínimas (RNF-09)	● En inicio

Documento generado durante la ejecución del plan · SalesIA Enterprise · Versión 1.0